T3000 API & Protocol

API Reference Overview · built Nov 13, 2026

API Reference Overview

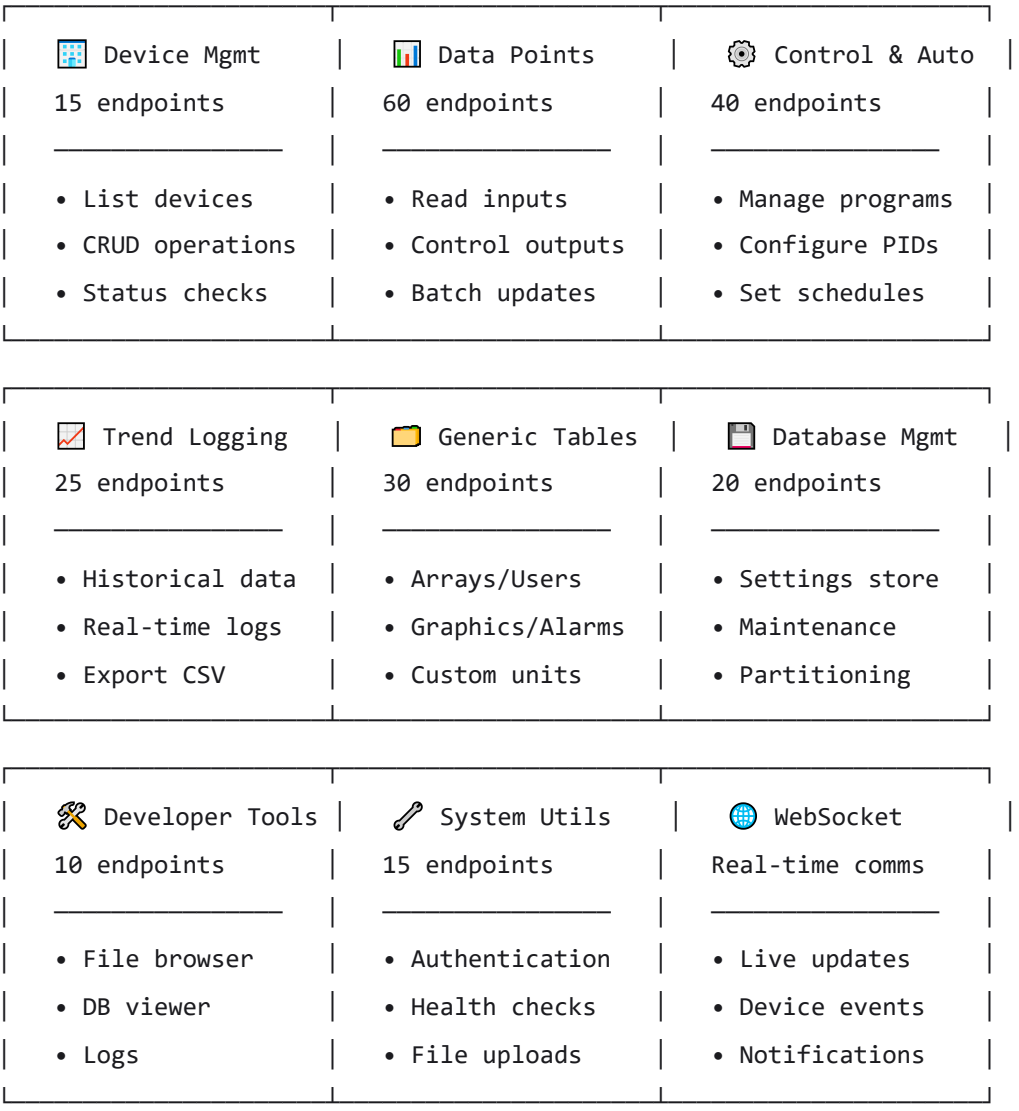
Introduction

The T3000 WebView provides a comprehensive REST API for building automation and device management. This overview lists all **215+ available endpoints** organized by category.

Base URL: `http://localhost:9103`

Quick Navigation

TEXT



Complete API Index

🏢 Device Management (15 endpoints)

```
GET    /api/t3_device/devices ............................... List all devices
GET    /api/t3_device/devices/:id ........................... Get device by ID
POST   /api/t3_device/devices ............................... Create new device
PUT    /api/t3_device/devices/:id ........................... Update device
DELETE /api/t3_device/devices/:id ........................... Delete device
GET    /api/t3_device/devices/count ......................... Get device count
GET    /api/t3_device/devices/:id/capabilities .............. Get capabilities
POST   /api/t3_device/devices/:id/refresh ................... Refresh from hardware
GET    /api/t3_device/devices/:id/connection-status ......... Check connection
```

Quick Example:

```
GET /api/t3_device/devices
→ Returns: [{ id: 1, serial_number: 123456, product_name: "T3-BB", online: true }]
```

📊 Data Points (60 endpoints)

Inputs (20 endpoints)

```
GET    /api/t3_device/devices/:id/input-points .............. Get all inputs
GET    /api/t3_device/inputs/:objectinstance ................ Get single input
PUT    /api/t3_device/inputs/:objectinstance ................ Update input
POST   /api/t3_device/devices/:id/inputs/refresh ............ Refresh inputs
GET    /api/t3_device/inputs/:objectinstance/history ........ Input history
```

Outputs (20 endpoints)

```
GET    /api/t3_device/devices/:id/output-points ..... Get all outputs
GET    /api/t3_device/outputs/:objectinstance ....... Get single output
PUT    /api/t3_device/outputs/:objectinstance ....... Update output
PUT    /api/t3_device/outputs/batch ................. Batch update outputs
POST   /api/t3_device/devices/:id/outputs/refresh ... Refresh outputs
GET    /api/t3_device/outputs/:objectinstance/history ..... Output history
```

Variables (20 endpoints)

```
GET    /api/t3_device/devices/:id/variable-points ..... Get all variables
GET    /api/t3_device/variables/:objectinstance ....... Get single variable
PUT    /api/t3_device/variables/:objectinstance ....... Update variable
PUT    /api/t3_device/variables/batch ................. Batch update variables
POST   /api/t3_device/devices/:id/variables/refresh ... Refresh variables
GET    /api/t3_device/variables/:objectinstance/history ..... Variable history
```

Quick Example:

```
PUT /api/t3_device/outputs/5000000
{ "value": 75.0, "auto_manual": 1 }
→ Sets output to 75% in manual mode
```

⚙ Control & Automation (40 endpoints)

Programs (10 endpoints)

```
GET    /api/t3_device/devices/:id/programs ..... Get all programs
GET    /api/t3_device/programs/:id ............. Get single program
POST   /api/t3_device/programs ................. Create program
PUT    /api/t3_device/programs/:id ............. Update program
DELETE /api/t3_device/programs/:id ............. Delete program
POST   /api/t3_device/programs/:id/compile ..... Compile program
POST   /api/t3_device/programs/:id/start ....... Start program
POST   /api/t3_device/programs/:id/stop ........ Stop program
```

Schedules (10 endpoints)

```
GET    /api/t3_device/devices/:id/schedules ..... Get all schedules
GET    /api/t3_device/schedules/:id ............. Get single schedule
POST   /api/t3_device/schedules ................. Create schedule
PUT    /api/t3_device/schedules/:id ............. Update schedule
DELETE /api/t3_device/schedules/:id ............. Delete schedule
POST   /api/t3_device/schedules/:id/enable ..... Enable schedule
POST   /api/t3_device/schedules/:id/disable .... Disable schedule
```

PIDs (10 endpoints)

```
GET    /api/t3_device/devices/:id/pids ......... Get all PIDs
GET    /api/t3_device/pids/:id ................. Get single PID
POST   /api/t3_device/pids ..................... Create PID
PUT    /api/t3_device/pids/:id ................. Update PID tuning
DELETE /api/t3_device/pids/:id ................. Delete PID
POST   /api/t3_device/pids/:id/autotune ........ Auto-tune PID
```

Holidays (5 endpoints)

```
GET    /api/t3_device/devices/:id/holidays ..... Get all holidays
GET    /api/t3_device/holidays/:id ............. Get single holiday
POST   /api/t3_device/holidays ................. Create holiday
PUT    /api/t3_device/holidays/:id ............. Update holiday
DELETE /api/t3_device/holidays/:id ............. Delete holiday
```

Annual Routines (5 endpoints)

```
GET    /api/t3_device/devices/:id/annual-routines ..... Get all routines
GET    /api/t3_device/annual-routines/:id ..... Get single routine
POST   /api/t3_device/annual-routines ..... Create routine
PUT    /api/t3_device/annual-routines/:id ..... Update routine
DELETE /api/t3_device/annual-routines/:id ..... Delete routine
```

📈 Trend Logging (25 endpoints)

```
GET    /api/t3_device/devices/:id/trendlogs ..... Get all trendlogs
GET    /api/t3_device/trendlogs/:id ..... Get trendlog config
POST   /api/t3_device/trendlogs ..... Create trendlog
PUT    /api/t3_device/trendlogs/:id ..... Update trendlog
DELETE /api/t3_device/trendlogs/:id ..... Delete trendlog
GET    /api/t3_device/trendlogs/:id/data ..... Get historical data
POST   /api/t3_device/trendlogs/:id/clear ..... Clear trendlog data
GET    /api/t3_device/trendlogs/:id/export ..... Export to CSV
POST   /api/t3_device/trendlogs/:id/start ..... Start logging
POST   /api/t3_device/trendlogs/:id/stop ..... Stop logging
GET    /api/t3_device/trendlogs/:id/stats ..... Get statistics
```

Quick Example:

```
GET /api/t3_device/trendlogs/1/data?start=2026-01-01&end=2026-01-09
→ Returns: { data: [{ timestamp: "2026-01-09T08:00:00Z", value: 72.5 }] }
```

📁 Generic Tables (30 endpoints)

```
GET    /api/t3_device/:table ................................ Get table records
GET    /api/t3_device/:table/count ........................... Get record count
GET    /api/t3_device/:table/:id ............................. Get single record
PUT    /api/t3_device/:table/:id ............................. Update record
POST   /api/t3_device/:table ................................. Create record
DELETE /api/t3_device/:table/:id ............................. Delete record
```

Supported Tables:

- `arrays` - Array configuration
- `users` - User management
- `graphics` - Graphics screens
- `alarms` - Alarm definitions
- `units` - Unit conversions
- `conversions` - Custom conversions
- `custom_units` - Custom unit definitions

Quick Example:

```http
GET /api/t3_device/users?device_id=1
→ Returns: [{ id: 1, username: "admin", access_level: 255 }]
```

💾 Database Management (20 endpoints)

```
TEXT

GET    /api/db_management/settings ........................... Get all settings
POST   /api/db_management/settings ........................... Save setting
DELETE /api/db_management/settings/:key ...................... Delete setting
GET    /api/database/stats ................................... Database statistics
POST   /api/db_management/tools/vacuum ....................... Vacuum database
POST   /api/database/cleanup/old ............................. Clean old files
GET    /api/db_management/partitions ......................... Get partitions
POST   /api/db_management/partitions ......................... Create partition
DELETE /api/db_management/partitions/:name ................... Drop partition
GET    /api/database/config .................................. Get DB config
PUT    /api/database/config .................................. Update DB config
GET    /api/database/backup .................................. Create backup
POST   /api/database/restore ................................. Restore from backup
```

🛠️ Developer Tools (10 endpoints)

```
TEXT

GET    /api/develop/files/list ............................... List files
GET    /api/develop/files/read ............................... Read file
POST   /api/develop/files/write .............................. Write file
DELETE /api/develop/files/delete ............................. Delete file
GET    /api/develop/database/list ............................ List databases
POST   /api/develop/database/query ........................... Execute SQL query
GET    /api/develop/database/schema .......................... Get schema
GET    /api/develop/logs/get ................................. Get system logs
POST   /api/develop/logs/clear ............................... Clear logs
GET    /api/develop/logs/download ............................ Download log file
```

Quick Example:

```
HTTP

POST /api/develop/database/query
{ "sql": "SELECT * FROM DEVICES LIMIT 10" }
→ Returns: { columns: ["id", "serial_number"], rows: [[1, 123456]] }
```

🔧 System & Utilities (15 endpoints)

```
GET    /api/health ..................................... Health check
GET    /api/version .................................... API version
POST   /api/login ...................................... User login
POST   /api/logout ..................................... User logout
GET    /api/session .................................... Check session
POST   /api/upload ..................................... Upload file
GET    /api/uploads/:filename .......................... Download file
DELETE /api/uploads/:filename .......................... Delete file
GET    /api/modbus/registers ........................... Read Modbus registers
POST   /api/modbus/write ............................... Write Modbus register
POST   /api/ffi/call ................................... Call BACnet FFI
GET    /api/sync/metadata .............................. Get sync metadata
POST   /api/sync/start ................................. Start sync
POST   /api/sync/stop .................................. Stop sync
GET    /api/sync/status ................................ Sync status
```

Quick Example:

```http
POST /api/login
{ "username": "admin", "password": "password" }
→ Returns: { success: true, session_token: "abc123..." }
```

🌐 WebSocket (Real-time Communication)

```
WS     /ws ............................................. WebSocket connection
```

Message Types:

- `1` - SELECT_PANEL: Select active device
- `2` - UPDATE_POINT: Point value updated
- `3` - DEVICE_STATUS: Device online/offline
- `4` - REFRESH_COMPLETE: Refresh operation complete

Quick Example:

JAVASCRIPT

```javascript
const ws = new WebSocket('ws://localhost:9103/ws');
ws.send(JSON.stringify({ action: 1, panelId: 5, serialNumber: 123456 }));
```

Common Workflows

Workflow 1: Device Setup & Monitoring

TEXT

```
1. GET  /api/t3_device/devices              → Get all devices
2. GET  /api/t3_device/devices/1            → Get device details
3. GET  /api/t3_device/devices/1/input-points   → Read inputs
4. GET  /api/t3_device/devices/1/output-points  → Read outputs
5. WS   /ws (subscribe)                     → Live updates
```

Workflow 2: Control Operations

TEXT

```
1. GET  /api/t3_device/outputs/5000000      → Read current value
2. PUT  /api/t3_device/outputs/5000000      → Write new value
   { "value": 75.0, "auto_manual": 1 }
3. GET  /api/t3_device/outputs/5000000      → Verify change
```

Workflow 3: Historical Data Analysis

TEXT

```
1. GET  /api/t3_device/devices/1/trendlogs  → List trendlogs
2. GET  /api/t3_device/trendlogs/1/data     → Query data
   ?start=2026-01-01&end=2026-01-09
3. GET  /api/t3_device/trendlogs/1/export   → Export CSV
```

Workflow 4: Batch Control

```text
1. PUT  /api/t3_device/outputs/batch        → Update multiple outputs
   {
     "updates": [
       { "objectinstance": 5000000, "value": 50.0 },
       { "objectinstance": 5000001, "value": 75.0 }
     ]
   }
```

Authentication

Currently, the API uses basic session-based authentication.

```typescript
// Login
const response = await fetch('http://localhost:9103/api/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: 'admin',
    password: 'password'
  })
});
```

Response Format

Success Response

```json
{
  "success": true,
  "data": { ... },
  "message": "Operation completed successfully"
}
```

Error Response

```JSON
{
  "error": "Device not found",
  "code": "DEVICE_NOT_FOUND",
  "status": 404
}
```

Common HTTP Status Codes

- `200 OK` - Request successful
- `201 Created` - Resource created
- `400 Bad Request` - Invalid parameters
- `401 Unauthorized` - Authentication required
- `404 Not Found` - Resource not found
- `500 Internal Server Error` - Server error
- `503 Service Unavailable` - Database unavailable

CORS

CORS is enabled for all origins in development. Configure for production:

```RUST
CorsLayer::new()
    .allow_origin("https://your-domain.com")
    .allow_methods([Method::GET, Method::POST, Method::PUT, Method::DELETE])
```

Complete Endpoint Reference

Quick Reference Table

All **215+ endpoints** organized by HTTP method and category:

TEXT

```
┌────────┬──────────────────────────────────────────┬────────┬──────────────┐
│ Method │ Endpoint Pattern                         │ Auth   │ Category     │
├────────┼──────────────────────────────────────────┼────────┼──────────────┤
│ GET    │ /api/t3_device/devices                   │ ✓      │ Device       │
│ GET    │ /api/t3_device/devices/:id               │ ✓      │ Device       │
│ POST   │ /api/t3_device/devices                   │ ✓      │ Device       │
│ PUT    │ /api/t3_device/devices/:id               │ ✓      │ Device       │
│ DELETE │ /api/t3_device/devices/:id               │ ✓      │ Device       │
│ GET    │ /api/t3_device/devices/count             │ ✓      │ Device       │
│ GET    │ /api/t3_device/devices/:id/capabilities  │ ✓      │ Device       │
│ POST   │ /api/t3_device/devices/:id/refresh       │ ✓      │ Device       │
├────────┼──────────────────────────────────────────┼────────┼──────────────┤
│ GET    │ /api/t3_device/devices/:id/input-points  │ ✓      │ Data Points  │
│ GET    │ /api/t3_device/inputs/:objectinstance    │ ✓      │ Data Points  │
│ PUT    │ /api/t3_device/inputs/:objectinstance    │ ✓      │ Data Points  │
│ POST   │ /api/t3_device/devices/:id/inputs/refresh│ ✓      │ Data Points  │
│ GET    │ /api/t3_device/outputs/:objectinstance   │ ✓      │ Data Points  │
│ PUT    │ /api/t3_device/outputs/:objectinstance   │ ✓      │ Data Points  │
│ PUT    │ /api/t3_device/outputs/batch             │ ✓      │ Data Points  │
│ POST   │ /api/t3_device/devices/:id/outputs/refresh│ ✓     │ Data Points  │
│ GET    │ /api/t3_device/variables/:objectinstance│ ✓      │ Data Points  │
│ PUT    │ /api/t3_device/variables/:objectinstance│ ✓      │ Data Points  │
│ PUT    │ /api/t3_device/variables/batch           │ ✓      │ Data Points  │
├────────┼──────────────────────────────────────────┼────────┼──────────────┤
│ GET    │ /api/t3_device/programs                  │ ✓      │ Control      │
│ POST   │ /api/t3_device/programs                  │ ✓      │ Control      │
│ PUT    │ /api/t3_device/programs/:id              │ ✓      │ Control      │
│ DELETE │ /api/t3_device/programs/:id              │ ✓      │ Control      │
│ GET    │ /api/t3_device/schedules                 │ ✓      │ Control      │
│ POST   │ /api/t3_device/schedules                 │ ✓      │ Control      │
│ PUT    │ /api/t3_device/schedules/:id             │ ✓      │ Control      │
│ DELETE │ /api/t3_device/schedules/:id             │ ✓      │ Control      │
│ GET    │ /api/t3_device/pids                      │ ✓      │ Control      │
│ PUT    │ /api/t3_device/pids/:id                  │ ✓      │ Control      │
│ GET    │ /api/t3_device/holidays                  │ ✓      │ Control      │
│ PUT    │ /api/t3_device/holidays/:id              │ ✓      │ Control      │
├────────┼──────────────────────────────────────────┼────────┼──────────────┤
│ GET    │ /api/t3_device/trendlogs                 │ ✓      │ Trends       │
│ GET    │ /api/t3_device/trendlogs/:id/data        │ ✓      │ Trends       │
│ GET    │ /api/t3_device/trendlogs/:id/export      │ ✓      │ Trends       │
```

POST	/api/t3_device/trendlogs/:id/clear	✓	Trends
PUT	/api/t3_device/trendlogs/:id	✓	Trends
GET	/api/t3_device/:table	✓	Tables
GET	/api/t3_device/:table/count	✓	Tables
GET	/api/t3_device/users	✓	Tables
POST	/api/t3_device/users	✓	Tables
PUT	/api/t3_device/users/:id	✓	Tables
DELETE	/api/t3_device/users/:id	✓	Tables
GET	/api/db_management/settings	✓	Database
POST	/api/db_management/settings	✓	Database
DELETE	/api/db_management/settings/:key	✓	Database
GET	/api/database/stats	✓	Database
POST	/api/db_management/tools/vacuum	✓	Database
GET	/api/db_management/partitions	✓	Database
GET	/api/develop/files/list	✓	Developer
GET	/api/develop/files/read	✓	Developer
POST	/api/develop/database/query	✓	Developer
GET	/api/develop/database/schema	✓	Developer
GET	/api/develop/logs/get	✓	Developer
GET	/api/health	-	System
GET	/api/version	-	System
POST	/api/login	-	Auth
POST	/api/logout	✓	Auth
GET	/api/session	✓	Auth
POST	/api/upload	✓	System
GET	/api/uploads/:filename	✓	System
DELETE	/api/uploads/:filename	✓	System
GET	/api/modbus/registers	✓	System
POST	/api/modbus/write	✓	System
POST	/api/ffi/call	✓	System
WS	/ws	✓	WebSocket

Legend: ✓ = Authentication required | - = Public endpoint

API Coverage Matrix

Feature availability by resource type:

```
TEXT
```

```
┌──────────────────┬──────┬─────┬────────┬────────┬────────┬──────────┐
Resource          │ List │ Get │ Create │ Update │ Delete │ Special
├──────────────────┼──────┼─────┼────────┼────────┼────────┼──────────┤
Devices           │  ✓   │  ✓  │   ✓    │   ✓    │   ✓    │ Refresh
Inputs            │  ✓   │  ✓  │   -    │   ✓    │   -    │ Refresh
Outputs           │  ✓   │  ✓  │   -    │   ✓    │   -    │ Batch
Variables         │  ✓   │  ✓  │   -    │   ✓    │   -    │ Batch
Programs          │  ✓   │  ✓  │   ✓    │   ✓    │   ✓    │ Compile
Schedules         │  ✓   │  ✓  │   ✓    │   ✓    │   ✓    │ Enable
PIDs              │  ✓   │  ✓  │   ✓    │   ✓    │   ✓    │ Autotune
Holidays          │  ✓   │  ✓  │   ✓    │   ✓    │   ✓    │ -
Trendlogs         │  ✓   │  ✓  │   ✓    │   ✓    │   ✓    │ Export
Users             │  ✓   │  ✓  │   ✓    │   ✓    │   ✓    │ -
Settings          │  ✓   │  ✓  │   ✓    │   ✓    │   ✓    │ -
└──────────────────┴──────┴─────┴────────┴────────┴────────┴──────────┘


Special Operations:
• Batch Updates      → Outputs, Variables
• Refresh from HW    → Inputs, Outputs, Variables, Devices
• Export to CSV      → Trendlogs
• Real-time Stream   → WebSocket (/ws)
• SQL Queries        → Developer Tools
• File Management    → Developer Tools
```

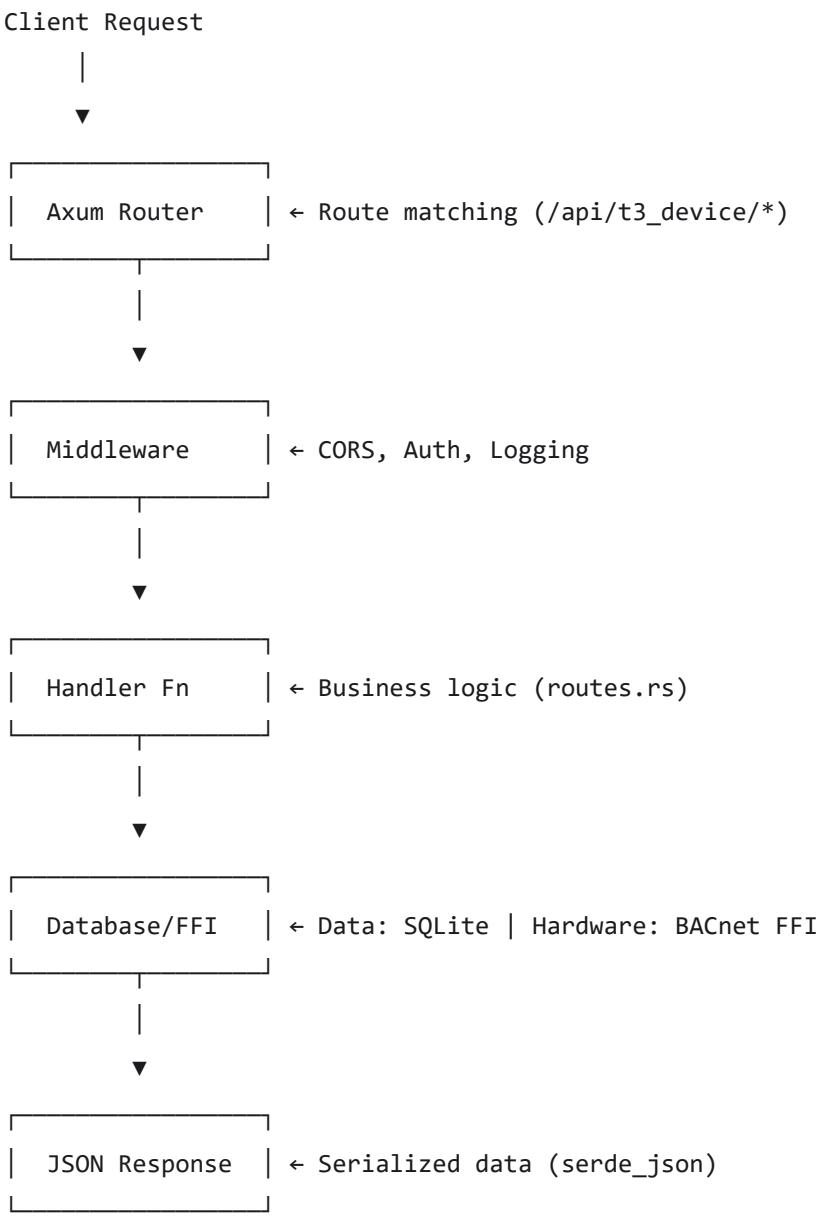
API Architecture

The T3000 WebView API is built with:

- **Framework**: Axum (Rust) - High-performance async web framework
- **Database**: SQLite via SeaORM - Type-safe ORM
- **WebSocket**: Real-time bidirectional communication
- **FFI**: BACnet protocol via C++ bridge

Request/Response Lifecycle

TEXT



Content Types

Request Headers:

HTTP

```
Content-Type: application/json
Accept: application/json
```

Response Headers:

```http
Content-Type: application/json; charset=utf-8
Access-Control-Allow-Origin: *
```

Rate Limiting

Currently no rate limiting is enforced. For production, implement:

```rust
use tower::limit::RateLimitLayer;

app.layer(RateLimitLayer::new(100, Duration::from_secs(60)))
```

WebSocket Connection

For real-time updates, connect via WebSocket:

```typescript
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  console.log('Connected to T3000 WebView');
};

ws.onmessage = (event) => {
  const message = JSON.parse(event.data);
  console.log('Received:', message);
};

// Send control message
ws.send(JSON.stringify({
  action: 1,  // SELECT_PANEL
  panelId: 5,
  serialNumber: 123456
}));
```

Error Handling

Client-Side Pattern:

TYPESCRIPT

```typescript
async function apiCall(endpoint: string, options?: RequestInit) {
  try {
    const response = await fetch(`http://localhost:9103${endpoint}`, options);

    if (!response.ok) {
      const error = await response.json();
      throw new Error(error.message || `HTTP ${response.status}`);
    }

    return await response.json();
  } catch (error) {
    console.error('API Error:', error);
    throw error;
  }
}
```

Server-Side Error Types:

RUST

```rust
pub enum ApiError {
    DatabaseError(String),
    DeviceOffline,
    InvalidRequest(String),
    NotFound(String),
    Unauthorized,
    InternalError(String),
}

impl IntoResponse for ApiError {
    fn into_response(self) -> Response {
        let (status, message) = match self {
            ApiError::NotFound(msg) => (StatusCode::NOT_FOUND, msg),
            ApiError::Unauthorized => (StatusCode::UNAUTHORIZED, "Unauthorized".into()),
            // ...
        };

        (status, Json(json!({ "error": message }))).into_response()
    }
}
```

Performance Considerations

Pagination:

```HTTP
GET /api/t3_device/devices?page=1&per_page=50
```

Field Selection (future):

```HTTP
GET /api/t3_device/devices/1?fields=id,serial_number,ip_address
```

Batch Operations:

```HTTP
POST /api/t3_device/inputs/batch
{
  "updates": [
    { "objectinstance": 3000000, "value": 72.5 },
    { "objectinstance": 3000001, "value": 65.0 }
  ]
}
```

API Versioning

Current API is v1 (implicit). Future versions:

```TEXT
/api/v2/t3_device/devices
```

Testing Endpoints

Using curl:

SHELL

```shell
## Get devices
curl http://localhost:9103/api/t3_device/devices

## Create device
curl -X POST http://localhost:9103/api/t3_device/devices \
  -H "Content-Type: application/json" \
  -d '{"serial_number": 123456, "product_name": "T3-BB"}'
```

Using JavaScript:

JAVASCRIPT

```javascript
// GET request
fetch('http://localhost:9103/api/t3_device/devices')
  .then(res => res.json())
  .then(data => console.log(data));

// POST request
fetch('http://localhost:9103/api/t3_device/devices', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    serial_number: 123456,
    product_name: 'T3-BB'
  })
})
  .then(res => res.json())
  .then(data => console.log(data));
```

Database Schema Reference

Key tables accessed by API:

- `DEVICES` - Device registry
- `INPUTS` - Analog/Binary inputs
- `OUTPUTS` - Analog/Binary outputs
- `VARIABLES` - Analog values
- `PROGRAMS` - Control programs
- `SCHEDULES` - Time schedules
- `TRENDLOGS` - Trendlog configuration

- `TRENDLOG_DATA` - Historical data

File Locations

- **Routes**: `api/src/t3_device/routes.rs`
- **Services**: `api/src/t3_device/*_service.rs`
- **State**: `api/src/app_state.rs`
- **Server**: `api/src/server.rs`

Development Setup

```shell
## Start backend
cd api
cargo run

## Backend runs on http://localhost:9103

## Test endpoint
curl http://localhost:9103/api/health
```

Production Deployment

1. Build optimized binary:

```shell
cargo build --release
```

2. Configure environment:

```env
DATABASE_URL=./production.db
PORT=9103
RUST_LOG=info
```

3. Run as service:

```shell
./target/release/t3000-api
```